Symbols

Often, when programming, you need to use the same string over and over. Perhaps the string is a key in a Hash, or maybe the string is the name of a method. In that case, you'd probably want access to that string to be immutable so its value can't change, and you also want accessing the string be as fast and use as little memory as possible.

Which brings us to Ruby's *symbols*. Symbols aren't exactly optimized strings, but for most purposes, you can think of them as special strings that are immutable, are only created once, and are very fast to look up. Symbols are meant to be used as keys and identifiers, where strings are meant to be used for data.

A symbol literal starts with a colon and is followed by some kind of name:

```
walk(:north)
look(:east)
```

In this example, we are using the symbols :north and :south to represent constant values in the code. We don't need to declare the symbols or assign them a value—Ruby takes care of that for you. The value of a symbol is equivalent to its name.

Ruby also guarantees that no matter where they appear in your program, symbols with the same name will have the same value—indeed, they will be the same internal object. As a result, you can safely write the following:

```
def walk(direction)
  if direction == :north
    # ...
  end
end
```

Because their values don't change, symbols are frequently used as keys in hashes. We could write our previous hash example using symbols as keys:

intro/hash_with_symbol_keys.rb

```
instrument_section = {
  :cello => "string",
  :clarinet => "woodwind",
  :drum => "percussion",
  :oboe => "woodwind",
  :trumpet => "brass",
  :violin => "string"
}
instrument_section[:oboe]    # => "woodwind"
instrument_section[:cello]   # => "string"
# Note that strings aren"t the same as symbols...
instrument_section["cello"]  # => nil
```

Note from the last line that a symbol key is different from a string key, and access via one will not result in a value associated with the other.

Symbols are so frequently used as hash keys that Ruby has a shortcut syntax: you can use name: value pairs to create a hash instead of name => value if the key is a symbol:

intro/hash_with_symbol_keys_19.rb

```
instrument_section = {
```